

VT_SMUTER — Version A

Phase-wise research workflow: image-encoded Vision Transformer with explainable AI for Buy / Hold / Sell classification on NIFTY 50 (90 indicators × 90 days)

Prepared for: Pio Micheal, Department of Artificial Intelligence and Data Science, Karunya Institute of Technology and Sciences | September 2026

Base paper: A. H. B. Gezici and E. Sefer, “Deep Transformer-Based Asset Price and Direction Prediction,” *IEEE Access*, vol. 12, pp. 24164–24178, 2024. Code: github.com/seferlab/price_transformer

1. Scope and fixed design decisions

Version A replicates the base paper's image-encoding and ViT pipeline on the Indian equity market, expands the indicator set from 65 to 90, adds explainable-AI analysis of the ViT's decisions, and fixes the evaluation defects found in the base paper. Every later extension (faithfulness benchmarking, permutation stability) can reuse the artifacts produced here without retraining, because checkpoints, predictions, and attribution maps are saved per fold.

Decision	Version A choice	Rationale
Universe	Current NIFTY 50 constituents with full price history over the study period, plus the NIFTY 50 index (^NSEI)	Data available via yfinance; survivorship bias declared as a limitation
Period	Jan 2008 – Dec 2025 (adjust start after availability check in Phase 2)	Covers 2008 crisis, 2020 COVID crash, post-2021 regime
Input image	90 indicators (rows) × 90 trading days (columns), single channel	Square image keeps base-paper structure
Row order	Grouped by TA-Lib category, fixed order frozen in config	Same principle as base paper; ordering is logged
Labels	Next-day return with threshold 0.01 (main); per-fold balanced threshold (secondary)	Mirrors base paper; balanced threshold fitted on train data only
Main model	ViT from scratch, 9×9 patches (100 tokens); variant with 3×9 row-narrow patches	9×9 for comparability; 3×9 keeps patches closer to indicator rows for XAI
Baselines	Majority-class, Buy & Hold, RSI(14), SMA(50), LSTM, CNN-TA++ (90×90)	Majority-class baseline added to expose inflated accuracy
Validation scheme	Walk-forward: 5-year train (last 6 months = validation), next 1-year test	Same as base paper, plus a proper validation split
XAI	Chefer relevance (primary), attention rollout, Integrated Gradients; top-k retraining check	Class-specific method plus a cheap test that explanations carry signal
Primary metrics	Macro-F1, per-class recall; Sharpe, max drawdown, CAGR after costs	Accuracy alone is misleading under Hold-dominant labels

2. Base-paper corrections and where they are enforced

Each defect found in the base paper maps to a concrete control in this workflow. The audit in Phase 1 confirms whether the defects originate in the released code before any reuse.

Issue in base paper	Correction in this workflow	Phase
Reported accuracies do not match confusion-matrix diagonals (e.g., ConvMixer 87.94% reported vs 31.8% recomputed; the reported figure equals the Hold-class share)	Accuracy computed only from saved predictions; automated assertion that $\text{accuracy} = \text{trace}(\text{CM}) / \text{sum}(\text{CM})$; majority-class baseline reported	1, 7
Backtest shows negative capital (Swin min capital −73.5M) and 100% success with negative return (DeiT)	Long-only engine with runtime invariants: $\text{cash} \geq 0$, $\text{shares} \geq 0$, $\text{equity} = \text{cash} + \text{shares} \times \text{price}$; unit tests on synthetic series	7
Test sample count (1,244) inconsistent with 15 test years × 9 ETFs	Per-fold sample counts logged and reconciled against calendar days × assets	4, 9

Issue in base paper	Correction in this workflow	Phase
Standardization scope unspecified (leakage risk)	Normalization statistics fitted on the training fold only; price-level indicators converted to relative form first	5
Label construction and threshold selection loosely described	Explicit label formula, 1-day embargo at fold boundaries, balanced threshold fitted per fold on training returns	4
Per-ETF accuracy table contradicts pooled accuracy	Pooled and per-asset metrics generated by one script from the same prediction files	7, 9
Pairwise t-tests without multiple-comparison correction	McNemar and block-bootstrap tests with Holm–Bonferroni correction; effect sizes reported	7
Flat \$1 commission, execution timing not stated	Signal at close t, execution at open t+1; proportional costs with 0 / 10 / 25 bps sensitivity	7
Single run per model, no variance reported	Three seeds per model per fold; mean $\pm$ standard deviation	6, 7

3. Workflow at a glance

Phase	Name	Duration	Exit gate (summary)
0	Environment and reproducibility setup	0.5 wk	Seeded smoke run logs commit, config, versions
1	Base-paper code audit	1 wk	Audit report: reported vs recomputed numbers
2	Data acquisition and cleaning	0.5 wk	Hashed raw data; quality report signed off
3	Indicator engineering (90)	1 wk	Indicator list frozen; causality test passes
4	Labels, folds, leakage controls	0.5 wk	Fold manifest and class distributions logged
5	Image generation and normalization	0.5 wk	Image pipeline tests pass; spot-check figures
6	Model training (ViT + baselines)	2 wk	All folds $\times$ seeds trained; checkpoints saved
7	Evaluation and backtesting	1.5 wk	Metric invariants pass; results tables generated
8	XAI analysis and top-k check	1.5 wk	Class-wise indicator rankings and top-k results
9	Final validation and reproducibility audit	1 wk	Clean-clone rerun reproduces a fold within tolerance
10	Write-up and optional dashboard	2 wk	Draft paper with all numbers script-generated

Total is about 12 weeks of work, which leaves roughly 4 weeks of buffer inside a 16-week plan. If the submission deadline is tighter, compress Phase 6 by retraining every two years instead of every year, and start writing the method section during Phase 6.

Phase 0 — Environment and reproducibility setup

Duration: 0.5 week | **Objective:** make every result traceable to a commit, a config, a data hash, and a seed.

Steps

- 0.1 Create a Git repository with the structure in Section 5. Keep data and large artifacts out of Git; track them with DVC or a SHA-256 manifest file.
- 0.2 Pin the environment (Python 3.11, PyTorch 2.x, timm, TA-Lib, yfinance, pandas, scikit-learn, Captum, pytest, MLflow) in a lock file. Record the TA-Lib C library version separately, since it is installed outside pip.
- 0.3 Write one seeding utility used by every script:

```
def seed_everything(seed):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = False
    torch.use_deterministic_algorithms(True, warn_only=True)
```
- 0.4 Drive every run from YAML configs (data, indicators, folds, model, training, backtest, XAI). No hard-coded hyperparameters in scripts.
- 0.5 Log per run: git commit, config snapshot, data manifest hash, library versions, GPU type, seed, wall-clock time. Store under results/<run_id>/.
- 0.6 Set up pytest and a single command (make test) that runs all unit tests from later phases.

Outputs

Repository skeleton, lock file, seeding utility, config templates, experiment-tracking setup, a smoke-test run on random data.

Exit gate: a dummy training run produces a results folder containing commit hash, config, versions, and seed, and a second run with the same seed gives identical metrics on CPU.

Phase 1 — Base-paper code audit

Duration: 1 week | **Objective:** confirm which base-paper defects come from the released code, and decide what can be reused safely.

Steps

- 1.1 Clone github.com/seferlab/price_transformer into audit/ (never into src/). Record the commit hash.
- 1.2 Run the image-generation and one model's training/evaluation on their ETF data, following their README. If the code does not run, document the failure and continue the audit by reading the code.
- 1.3 Recompute accuracy from their saved or regenerated predictions and compare with their reported metric and with the confusion-matrix diagonal.
- 1.4 Inspect the metric code: check whether accuracy is computed against the correct label vector, and whether averaging (micro, macro, weighted) matches the paper.
- 1.5 Inspect the backtest: check for negative cash, fractional or negative shares, re-buying with capital already invested, and how commission is applied.
- 1.6 Inspect preprocessing: where standardization statistics are fitted (global or per fold), how labels are shifted, and whether any indicator uses future data.
- 1.7 Count test samples per fold and compare with the paper's 1,244.
- 1.8 Write audit/AUDIT.md with a table of reported value, recomputed value, suspected cause, and the correction adopted in this workflow.

Outputs

AUDIT.md, a list of code components safe to reference (for example, indicator calculation logic), and a list of components to reimplement.

Exit gate: each defect listed in Section 2 is marked confirmed-in-code, not-reproducible, or not-in-code, with evidence. Nothing from audit/ is imported by src/.

Phase 2 — Data acquisition and cleaning

Duration: 0.5 week | **Objective:** build a clean, versioned daily OHLCV dataset for NIFTY 50 stocks.

Steps

- 2.1 Fetch daily OHLCV for current NIFTY 50 constituents (.NS tickers) and ^NSEI from yfinance. Save the raw download unchanged with the download date.
- 2.2 Check first available date per ticker. Keep only tickers with complete history over the chosen period; list excluded tickers and the reason.
- 2.3 Use split- and dividend-adjusted prices consistently (adjusted OHLC). Verify on two known corporate actions that there are no price jumps.
- 2.4 Quality checks: missing trading days against the NSE calendar, zero or negative prices, zero volume days, duplicated dates, and single-day moves beyond $\pm 20\%$ (flag, then verify manually).
- 2.5 Forward-fill only isolated single missing days; drop longer gaps and record them. Never interpolate across a gap that spans a label date.
- 2.6 Save cleaned data as Parquet per ticker and write the SHA-256 manifest.

Outputs

data/raw (untouched), data/interim (cleaned Parquet), data_quality_report.md, universe.csv with inclusion/exclusion reasons.

Exit gate: the quality report shows no unresolved flags, and the manifest hash is recorded in the config.

Phase 3 — Indicator engineering (90 indicators)

Duration: 1 week | **Objective:** compute a causal, justified set of 90 indicators that forms the image rows.

Steps

- 3.1 Start from the base paper's 65 TA-Lib indicators and add 25 through multi-horizon variants (for example RSI 7/14/21, ROC 5/10/20) and indicators omitted by the base paper (ATR, NATR, cycle indicators). Appendix A gives an indicative composition.
- 3.2 Drop candlestick-pattern functions: they are sparse integer flags and add little in a standardized image.
- 3.3 Convert price-level indicators into relative form, for example $\text{SMA}(20) / \text{Close} - 1$, $\text{BBANDS upper} / \text{Close} - 1$. This makes values comparable across stocks with different price levels, which the base paper's pooled standardization needs but does not describe.
- 3.4 Remove the warm-up period: drop the first $\max(\text{indicator lookback}) + 90$ days per ticker so that no image contains NaN warm-up values.
- 3.5 Causality unit test: indicators at day t must not change when prices after t are altered.

```
x_full = compute_indicators(prices)
x_cut  = compute_indicators(prices.iloc[:t+1])
assert np.allclose(x_full.iloc[:t+1], x_cut, equal_nan=True)
```
- 3.6 Check redundancy: compute the correlation matrix of the 90 indicators on the first training fold. Report highly correlated pairs ($|r| > 0.95$), but keep them unless they are exact duplicates. Redundancy is a result to discuss, and it matters for XAI interpretation.
- 3.7 Define a 65-indicator subset matching the base paper for the 65-vs-90 comparison in Phase 6.
- 3.8 Freeze the final ordered list in `configs/indicators.yaml` (name, TA-Lib function, parameters, category, row index).

Outputs

Indicator matrices per ticker (days $\times$ 90) in Parquet, `indicators.yaml`, correlation heatmap, redundancy table.

Exit gate: the causality test passes for every indicator, no NaNs remain after warm-up removal, and the list is frozen and committed.

Phase 4 — Labels, walk-forward folds, and leakage controls

Duration: 0.5 week | **Objective:** define targets and splits so that no information from the future enters training.

Steps

- 4.1 Label for the sample ending on day t , using closing prices:

```
r(t+1) = Close(t+1) / Close(t) - 1
Buy  if r(t+1) >  theta
Sell if r(t+1) < -theta
Hold otherwise      (theta = 0.01 main)
```
- 4.2 Balanced variant: for each fold, choose θ from the training-period returns so the three classes are roughly equal. Never fit θ on the full dataset.
- 4.3 Folds: training on 5 calendar years, validation on the last 6 months of that window, test on the following year. Roll forward one year. With 2008–2025 data and the warm-up removed, test years run from about 2013 to 2025.
- 4.4 Embargo: drop the last training sample and the last validation sample of each segment, since their labels use the first day of the next segment.
- 4.5 Windows may start inside the previous segment (images use the past 90 days). That is allowed, because only past features are used, and it is stated in the paper.
- 4.6 Log per fold and per split: date ranges, number of samples, number of tickers, class counts and percentages.

Outputs

labels.parquet, folds.yaml (fold manifest with exact dates), class_distribution.csv.

Exit gate: a leakage test confirms that for every training sample the label date is strictly before the first validation date, and class distributions are logged for every fold.

Phase 5 — Image generation and normalization

Duration: 0.5 week | **Objective:** produce 90×90 standardized images efficiently and verifiably.

Steps

- 5.1 Do not store every image. Store each ticker's indicator matrix once and build the 90-day window on the fly in a PyTorch Dataset. This keeps storage small and makes row masking (Phase 8) and ordering changes trivial.
- 5.2 For each fold, fit per-indicator mean and standard deviation on training-period values pooled across tickers. Apply the same statistics to validation and test. Clip standardized values to ± 5 .
- 5.3 Save normalization statistics per fold (norm_stats_fold_k.npz) so any image can be rebuilt exactly.
- 5.4 Image layout: row i = indicator i in the frozen order, column j = day $t - 89 + j$ (oldest on the left). Store as float32 with shape (1, 90, 90).
- 5.5 Unit tests: correct shape, no NaN or inf, rows match the config order, the rightmost column equals indicator values on day t , and test-fold statistics are never computed from test data.
- 5.6 Render spot-check figures of Buy, Hold, and Sell images for three tickers (similar to base paper Figure 1) with row-category bands marked.

Outputs

Dataset class, normalization statistics per fold, spot-check figures, passing tests.

Exit gate: all image tests pass and a random image rebuilt from raw data matches the Dataset output exactly.

Phase 6 — Model training: ViT and baselines

Duration: 2 weeks | **Objective:** train all models under identical splits, seeds, and budgets.

Steps

6.1 ViT configuration (from scratch, timm VisionTransformer):

```
img_size=90, in_chans=1, num_classes=3
patch_size=9      -> 10 x 10 = 100 tokens (main)
patch_size=(3,9)  -> 30 x 10 = 300 tokens (row-narrow variant)
embed_dim=192, depth=6, num_heads=6, mlp_ratio=4, drop_rate=0.1
```

6.2 Training: AdamW (lr 3e-4, weight decay 0.05), cosine schedule with warm-up, batch size 256, up to 50 epochs, early stopping on validation macro-F1 (patience 8), class-weighted cross-entropy using training-fold class frequencies, mixed precision.

6.3 Tune a small grid (learning rate, depth, dropout) on fold 1 validation only, then freeze the configuration for all folds. Report the grid in the appendix.

6.4 CNN-TA++: two 3×3 convolution layers, max pooling, dropout 0.25 and 0.5, fully connected output, on the same 90×90 images, trained with the same loss, early stopping, and seeds.

6.5 LSTM: the same 90 indicators as a sequence of 90 time steps (no image), two layers, hidden size 64, same training protocol. This separates the effect of the image encoding from the effect of the features.

6.6 Rule baselines (no training): majority class (always Hold), Buy & Hold, RSI(14) with 30/70 thresholds, SMA(50) crossover.

6.7 Ablations kept small: ViT on 65 indicators vs 90 indicators; ViT 9×9 vs 3×9 patches. Run on all folds but with one seed if compute is tight.

6.8 Run 3 seeds (0, 1, 2) per model per fold. Save best checkpoint, training curves, and test-set predictions with probabilities (predictions_fold_k_seed_s.parquet: date, ticker, true label, predicted label, three probabilities).

6.9 Monitor per-class validation recall during training. If Buy or Sell recall stays at zero, the model has collapsed to Hold; log it rather than silently re-tuning on test data.

Outputs

Checkpoints, training logs, prediction files for every model × fold × seed, tuning grid record.

Exit gate: every planned run has a prediction file, no run was tuned on test data, and the run registry lists all runs including failed ones.

Phase 7 — Evaluation and backtesting

Duration: 1.5 weeks | **Objective:** report classification and trading performance that is internally consistent and statistically honest.

Steps

7.1 Classification metrics from prediction files only, with automated invariants:

```
cm = confusion_matrix(y_true, y_pred, labels=[HOLD, BUY, SELL])
assert cm.sum() == len(y_true)
assert np.isclose(accuracy_score(y_true, y_pred), np.trace(cm) / cm.sum())
```

7.2 Report macro-F1 (primary), per-class precision / recall / F1, balanced accuracy, accuracy, and confusion matrices per fold and pooled; mean ± standard deviation across seeds. Always show the majority-class baseline next to accuracy.

7.3 Backtest rules per ticker: long-only, no leverage, no shorting. A Buy signal opens a full position if flat; a Sell signal closes it if long; Hold does nothing; repeated signals are ignored (same as base paper). Signal known at close t, trade at open t+1 (primary); trade at close t+1 as a robustness check.

7.4 Costs: proportional per side, reported at 0, 10, and 25 bps. Whole shares only; leftover cash stays in cash.

7.5 Portfolio: equal capital per ticker sleeve, aggregated daily. Runtime invariants on every day:

```
assert cash >= -1e-9 and shares >= 0
assert abs(equity - (cash + shares * price)) < 1e-6
```

7.6 Trading metrics: CAGR, annualized Sharpe (daily excess return over the 91-day Indian T-bill, $\times \sqrt{252}$), Sortino, maximum drawdown, Calmar, trades per year, win rate, average holding period, time in market.

7.7 Random-signal benchmark: 1,000 simulations with random signals matching each model's trade frequency. Report where the ViT's Sharpe falls in that distribution.

7.8 Statistics: McNemar test on paired predictions (ViT vs each learned baseline); moving-block bootstrap confidence intervals for Sharpe and CAGR differences; Holm–Bonferroni correction across comparisons. Report effect sizes, not only p-values.

7.9 Regime breakdown: results for 2013–2019, 2020–2021, and 2022–2025, reported descriptively.

7.10 Backtest engine unit tests on synthetic prices: a constant price gives zero return minus costs, a known up-trend with one Buy gives the analytic return, and an alternating signal sequence gives the expected trade count.

Outputs

Script-generated tables (CSV and LaTeX): classification results, trading results by cost level, statistical tests, regime breakdown; equity-curve and drawdown figures.

Exit gate: all invariants pass on every run, every table is regenerated by one command from prediction files, and the numbers in the tables agree with each other.

Phase 8 — XAI analysis and top-k check

Duration: 1.5 weeks | **Objective:** explain which indicators and which recent days drive Buy and Sell decisions, and test whether those explanations carry predictive signal.

Steps

- 8.1 Methods: Chefer et al. (2021) transformer relevance propagation as the primary class-specific method; attention rollout (Abnar and Zuidema, 2020) for comparison; Integrated Gradients (Captum) with the all-zero image as baseline (zero equals the training mean after standardization). Apply Grad-CAM to CNN-TA++ for a reference.
- 8.2 Sample: per fold, up to 1,000 test images per predicted class, stratified into correct and incorrect predictions. Use the seed-0 model; repeat on seeds 1 and 2 for a stability check.
- 8.3 Upsample token-level relevance maps to 90×90. Normalize each map to sum to 1 so samples can be averaged.
- 8.4 Aggregate: sum over columns gives indicator importance (90 values); sum within category gives category importance; sum over rows gives a recency profile across the 90 days.
- 8.5 Report class-wise rankings: top indicators and categories for Buy, Sell, and Hold, for correct vs incorrect predictions, and across the three market regimes.
- 8.6 Stability and agreement: Kendall's tau between indicator rankings across folds and seeds; Spearman correlation between rankings from the three methods. Report disagreement openly; it is a finding.
- 8.7 Model-randomization sanity check (Adebayo et al., 2018): randomize the ViT's weights and recompute explanations. If the maps barely change, that method is not reflecting what the model learned.
- 8.8 Top-k retraining check: rank indicators using explanations on each fold's validation set only (never test). Retrain the ViT with all rows except the top-k set to zero, for $k = 15, 30, 45$. Compare against five random-k masks and the full 90-row model on test macro-F1 and Sharpe.
- 8.9 Figures: class-average relevance heatmaps with category bands and indicator labels, category bar charts per class, recency profiles, and a few annotated single-image examples.

Outputs

Saved attribution maps (compressed arrays per fold), ranking tables, stability and agreement statistics, top-k results table, XAI figures.

Exit gate: top-k XAI-selected indicators are compared against random-k masks on every fold, and each figure is produced by a script from saved attribution arrays.

Phase 9 — Final validation and reproducibility audit

Duration: 1 week | **Objective:** prove the results can be regenerated and that every reported claim is supported.

Steps

- 9.1 Clean-clone rerun: fresh environment from the lock file, download or restore data by manifest hash, run fold 1 end-to-end for the ViT and CNN-TA++ with seed 0. Metrics must match within ± 0.005 macro-F1 (GPU non-determinism tolerance).
- 9.2 Run the full test suite: causality, leakage, image, metric invariants, backtest invariants.
- 9.3 Reconcile sample counts: per fold, trading days $\times$ tickers minus warm-up and embargo must equal the number of predictions.
- 9.4 Regenerate every table and figure from scratch with one command and diff against the versions in the draft.
- 9.5 Claims checklist: for each sentence in the results and conclusion that states a number or a comparison, record the table or figure that supports it and whether it survives the statistical correction.
- 9.6 Compare with the base paper only qualitatively (direction of findings), since market, period, and indicator set differ. Cite the audit for any numeric disagreement.

9.7 Prepare the release: code, configs, manifests, fold definitions, and a README with exact reproduction commands. Do not redistribute raw vendor data; provide the download script instead.

Outputs

Reproducibility report, claims checklist, release-ready repository with README.

Exit gate: the clean-clone rerun matches within tolerance, all tests pass, and every claim in the draft maps to a generated table or figure.

Phase 10 — Write-up and optional dashboard

Duration: 2 weeks | **Objective:** turn the verified results into a thesis chapter and a conference-length paper.

Steps

- 10.1 Paper outline (IEEE conference template): introduction; related work (CNN-TA, DAPP/DPPP and Gezici–Sefer, Grad-CAM on indicator images for KOSPI 200, ViT XAI methods); data and image encoding; models; evaluation protocol with the base-paper corrections; results; XAI analysis; limitations; conclusion.
- 10.2 Include a short, factual reproducibility note on the base paper, citing the audit. Keep the tone neutral.
- 10.3 Limitations to state: survivorship bias from current constituents, yfinance data quality, simplified cost model, the arbitrary nature of row ordering, and known limits of attention-based explanations.
- 10.4 Future work section sets up Version B: quantitative faithfulness benchmarking and row-permutation stability of explanations.
- 10.5 Optional Streamlit dashboard: reads saved predictions, equity curves, and attribution maps only (no training in the app). Views: select ticker and date, show the 90×90 image, model probabilities, relevance heatmap, top indicators, and equity curve.
- 10.6 Final supervisor review, then a pre-submission check against the target venue's page limit and formatting rules.

Outputs

Thesis chapter, paper draft, figures and tables, optional dashboard, submission package.

Exit gate: supervisor sign-off on the draft, and the repository tag matches the version used for all reported numbers.

5. Repository structure

```

vt_smutter/
  configs/          base.yaml data.yaml indicators.yaml folds.yaml
                    model_vit.yaml model_cnnta.yaml model_lstm.yaml
                    backtest.yaml xai.yaml
  data/             raw/ interim/ processed/      (not in Git; manifest.sha256)
  audit/            price_transformer/ (cloned)  AUDIT.md
  src/
    data/           download.py clean.py quality.py
    features/       indicators.py
    labels/         labels.py folds.py
    images/         dataset.py normalize.py
    models/         vit.py cnn_ta.py lstm.py rules.py
    train/          train.py seed.py registry.py
    eval/           metrics.py stats.py tables.py
    backtest/       engine.py portfolio.py random_benchmark.py
    xai/            chefer.py rollout.py integrated_gradients.py
                    aggregate.py topk.py sanity.py
  tests/           test_causality.py test_leakage.py test_images.py
                    test_metrics.py test_backtest.py
  scripts/         run_fold.py run_all.sh make_tables.py make_figures.py
  results/<run_id>/ config.yaml env.txt commit.txt checkpoint.pt
                    predictions.parquet attributions.npz
  dashboard/       app.py (optional)
  paper/           tables/ figures/ main.tex
  environment.lock Makefile README.md

```

6. Reproducibility checklist

Item	Requirement	Checked in
Code version	Every result folder stores the git commit; paper numbers come from a tagged release	0, 9
Environment	Lock file plus TA-Lib C version; clean-clone rerun documented	0, 9
Data	Raw download kept unchanged; SHA-256 manifest; download script with dates	2
Features	Frozen ordered indicator list; causality test	3
Splits	Fold manifest with exact dates; embargo; theta fitted per fold on training data	4
Normalization	Statistics fitted per fold on training data and saved	5
Randomness	Three seeds; deterministic settings; variance reported	0, 6
Tuning	Grid on fold 1 validation only; full grid reported	6
Metrics	Computed from saved predictions; invariant assertions	7
Backtest	Stated execution timing and costs; runtime invariants; synthetic tests	7
Statistics	Paired tests, bootstrap intervals, Holm–Bonferroni correction	7
XAI	Rankings from validation data only; sanity check; saved attribution arrays	8
Reporting	All tables and figures regenerated by script; claims checklist	9

7. Risks and mitigations

Risk	Mitigation
Base-paper code does not run or differs from the paper	Audit from code reading and paper tables; reimplement cleanly; document in AUDIT.md
yfinance gaps or errors for .NS tickers	Quality report in Phase 2; cross-check a sample of days against NSE bhavcopy files; exclude unreliable tickers
Compute budget (13 folds × models × 3 seeds)	Small ViT from scratch; retrain every two years if needed; single seed for ablations
ViT collapses to predicting Hold	Class-weighted loss, per-class recall monitoring, balanced-threshold variant; report collapse honestly

Risk	Mitigation
ViT does not beat CNN-TA++	Still publishable as a corrected replication with XAI findings; frame the contribution accordingly
Explanations are noisy or methods disagree	Aggregate to indicator and category level; report agreement statistics; rely on the top-k check for evidence
Schedule slip	4-week buffer; write method sections during training; dashboard is optional and last

Appendix A — Indicative 90-indicator composition

Counts are a starting point. The exact list, parameters, and order are frozen in configs/indicators.yaml during Phase 3.

Category (row block)	Count	Examples (parameters)
Overlap studies (relative to close)	18	SMA 10/20/50, EMA 10/20/50, WMA 20, DEMA 20, TEMA 20, TRIMA 20, KAMA 20, MIDPOINT 14, MIDPRICE 14, BBANDS upper/middle/lower 20, HT_TRENDLINE, SAR
Momentum	38	RSI 7/14/21, ROC 5/10/20, MOM 10/20, CMO 14, WILLR 14/28, CCI 14/20, MACD / signal / histogram, PPO, APO, ADX 14, ADXR, DX, PLUS_DI, MINUS_DI, AROON up/down, AROONOSC, STOCH K/D, STOCHF K/D, STOCHRSI K/D, MFI 14, TRIX 15, ULTOSC, BOP
Volume	6	OBV (differenced), AD, ADOSC 3/10, volume ratio to 20-day mean, MFI 28
Volatility	8	ATR 14/28, NATR 14/28, TRANGE, rolling std of returns 10/20, BBANDS width 20
Price transform	4	AVGPRICE, MEDPRICE, TYPPRICE, WCLPRICE (all relative to close)
Statistics and cycle	16	BETA vs ^NSEI 20, CORREL vs ^NSEI 20, LINEARREG / ANGLE / SLOPE / INTERCEPT 14, TSF 14, STDDEV 14, VAR 14, HT_DCPERIOD, HT_DCPHASE, HT_PHASOR in-phase/quadrature, HT_SINE sine/lead, HT_TRENDMODE
Total	90	

Appendix B — Key references for the workflow

- A. H. B. Gezici and E. Sefer, “Deep Transformer-Based Asset Price and Direction Prediction,” IEEE Access, vol. 12, 2024.
- O. B. Sezer and A. M. Ozbayoglu, “Algorithmic financial trading with deep convolutional neural networks: Time series to image conversion approach,” Applied Soft Computing, vol. 70, 2018.
- T. Tuncer, U. Kaya, E. Sefer, O. Alacam, and T. Hoser, “Asset price and direction prediction via deep 2D transformer and convolutional neural networks,” ACM ICAIF, 2022.
- “Combining CNN and Grad-CAM for profitability and explainability of investment strategy: Application to the KOSPI 200 futures,” Expert Systems with Applications, 2023.
- A. Dosovitskiy et al., “An image is worth 16×16 words: Transformers for image recognition at scale,” ICLR, 2021.
- H. Chefer, S. Gur, and L. Wolf, “Transformer interpretability beyond attention visualization,” CVPR, 2021.
- S. Abnar and W. Zuidema, “Quantifying attention flow in transformers,” ACL, 2020.
- M. Sundararajan, A. Taly, and Q. Yan, “Axiomatic attribution for deep networks,” ICML, 2017.
- J. Adebayo et al., “Sanity checks for saliency maps,” NeurIPS, 2018.